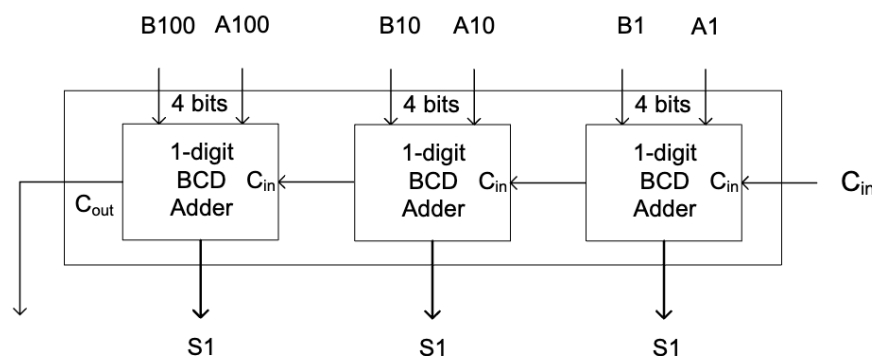




خلاصه

در این آزمایش یک جمع کننده سه رقمی (3-digit BCD Adder) در نرم افزار Logisim طراحی و پیاده سازی شده است. مدار به صورت سلسله مراتبی و از پایین به بالا ساخته شده: یک تمام جمع کننده ۱ بیتی (full_adder)، که پایه ی ساخت یک جمع کننده یک رقمی دهدهی با تصحیح خودکار BCD (bcd_digit) است، و در نهایت سه نمونه از این جمع کننده رقمی برای ساخت یک جمع کننده سه رقمی کامل زنجیر شده اند.

دیاگرام یک جمع کننده دهدهی



هدف آزمایش

هدف از این آزمایش، آشنایی عملی با طراحی مدارهای منطقی ترکیبی سلسله مراتبی و پیاده سازی یک جمع کننده اعداد دهدهی کد شده به صورت BCD (Binary Coded Decimal) است. برخلاف جمع باینری معمولی، جمع دو رقم BCD به یک مرحله ی تصحیح اضافه نیاز دارد تا نتیجه در بازه ی مجاز (۰ تا ۹) باقی بماند. اهداف مشخص این آزمایش عبارت اند از:

- پیاده سازی یک تمام جمع کننده ۱ بیتی (Full Adder) با گیت های پایه.
- طراحی مدار تصحیح BCD برای جمع دو رقم دهدهی به همراه رقم نقلی ورودی.
- ساخت یک جمع کننده سه رقمی با زنجیر کردن سه نمونه از جمع کننده یک رقمی.

کدگذاری BCD

در کدگذاری BCD، هر رقم دهدهی (۰ تا ۹) با ۴ بیت باینری نمایش داده می شود. برخلاف نمایش باینری خالص، در BCD کدهای ۱۰۱۰ تا ۱۱۱۱ (معادل ۱۰ تا ۱۵) بلا استفاده و نامعتبر هستند. هر رقم باید همیشه بین ۰۰۰۰ تا ۱۰۰۱ باشد.

جمع دو رقم BCD

اگر دو رقم BCD (هرکدام حداکثر ۹) به همراه یک رقم نقلی ورودی (حداکثر ۱) با یک جمع کننده باینری معمولی ۴ بیتی جمع زده شوند، نتیجه‌ی خام می‌تواند تا:

$$9 + 9 + 1 = 19$$

برسد که در بازه‌ی معتبر BCD (۰ تا ۹) نیست. اگر جمع خام از ۹ بیشتر باشد، باید مقدار ۶ (۰۱۱۰۰۲) به آن اضافه شود تا:

۱. رقم باقی مانده در بازه‌ی صحیح BCD قرار بگیرد.

۲. یک رقم نقلی دهدهی (نه باینری) به رقم بعدی منتقل شود.

تمام جمع کننده‌ی ۱ بیتی

پایه	جهت	پهنا
Cin, B, A	ورودی	۱ بیت
Cout, Sum	خروجی	۱ بیت

مدار با ۲ گیت XOR، ۲ گیت AND و ۱ گیت OR ساخته شده است:

$$X_1 = A \oplus B$$

نیمه جمع، بدون در نظر گرفتن رقم نقلی ورودی.

$$P_1 = A \wedge B$$

آیا از جمع A و B به تنهایی رقم نقلی تولید می‌شود؟

$$\text{Sum} = X_1 \oplus C_{in}$$

جمع نهایی سه ورودی.

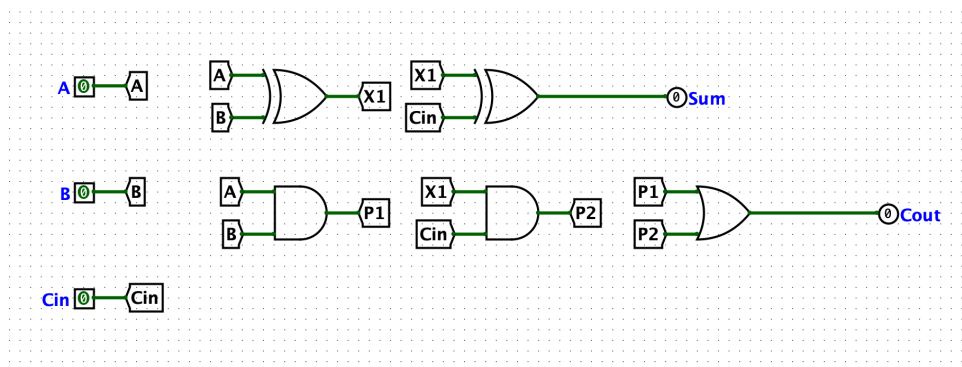
$$P_2 = X_1 \wedge C_{in}$$

آیا از جمع X_1 و C_{in} رقم نقلی تولید می‌شود؟

$$\text{Cout} = P_1 \vee P_2$$

رقم نقلی خروجی نهایی.

A	B	Cin	Sum	Cout
۰	۰	۰	۰	۰
۰	۰	۱	۱	۰
۰	۱	۰	۱	۰
۰	۱	۱	۰	۱
۱	۰	۰	۱	۰
۱	۰	۱	۰	۱
۱	۱	۰	۰	۱
۱	۱	۱	۱	۱



جمع‌کننده‌ی یک‌رقمی دهنده‌ی

پایه	جهت	پهنا
B, A	ورودی	۴ بیت (رقم BCD, ۰ تا ۹)
Cin	ورودی	۱ بیت
S	خروجی	۴ بیت (رقم نتیجه, ۰ تا ۹)
Cout	خروجی	۱ بیت (رقم نقلی دهنده‌ی)

این مدار در سه مرحله کار می‌کند که در ادامه هرکدام به‌طور کامل توضیح داده می‌شوند.

مرحله‌ی اول: جمع باینری خام. ورودی‌های A و B با یک تفکیک‌کننده به بیت‌های تکی $A_0...A_3$ و $B_0...B_3$ شکسته می‌شوند و در زنجیره‌ی اول از ۴ تمام‌جمع‌کننده (FA0 تا FA3) به‌صورت ripple-carry جمع زده می‌شوند:

نمونه	بیت	ورودی‌ها	خروجی جمع	رقم نقلی
FA0	۰ (کم‌ارزش)	A_0, B_0, C_{in}	S_0	CY_0
FA1	۱	A_1, B_1, CY_0	S_1	CY_1
FA2	۲	A_2, B_2, CY_1	S_2	CY_2
FA3	۳ (پرارزش)	A_3, B_3, CY_2	S_3	C_{outH}

نتیجه‌ی این مرحله عدد ۴ بیتی $S_3S_2S_1S_0$ (جمع باینری خام، هنوز تصحیح نشده) به‌همراه C_{outH} (رقم نقلی خروجی این جمع ۴ بیتی، معادل وزن ۱۶) است.

مرحله‌ی دوم: تشخیص نیاز به تصحیح. باید بررسی شود که آیا عدد $S_3S_2S_1S_0$ (با احتساب C_{outH}) از ۹ بزرگ‌تر است یا نه. چون C_{outH} به‌تنهایی فقط سرریز ۱۶ را نشان می‌دهد، فرمول کمینه‌ی زیر برای پوشش کامل بازه‌ی ۱۰ تا ۱۹ استفاده شده است:

$$CORR = C_{outH} \vee (S_3 \wedge S_2) \vee (S_3 \wedge S_1)$$

این فرمول با تنها ۳ گیت پیاده‌سازی شده است:

سیگنال	گیت	فرمول	معنی
G1OUT	OR	$S_2 \vee S_1$	آیا یکی از بیت‌های ۱ یا ۲ روشن است؟
G2OUT	AND	$S_3 \wedge G1OUT$	آیا بیت ۳ و یکی از بیت‌های ۱/۲ هم‌زمان روشن‌اند؟
CORR	OR	$C_{outH} \vee G2OUT$	سیگنال نهایی تصحیح

سیگنال CORR مستقیماً به پایه خروجی Cout این مدار وصل است؛ یعنی رقم نقلی دهدهی، همان سیگنال تصحیح است.

چرا این فرمول درست کار می‌کند؟

اگر $C_{outH} = 1$ جمع خام حتماً $9 < 16 \geq$ است. اگر $C_{outH} = 0$ جمع خام حداکثر ۱۵ است؛ در این حالت باید $S_3 = 1$ باشد تا مقدار $8 \geq$ شود. با $S_3 = 1$ اگر $S_2 = 1$ مقدار حداقل ۱۲ است (همیشه $9 >$)؛ اگر $S_2 = 0$ ولی $S_1 = 1$ مقدار حداقل ۱۰ است (باز هم $9 >$). این دقیقاً همان چیزی است که عبارت $(S_3 \wedge S_2) \vee (S_2 \wedge S_1)$ تشخیص می‌دهد.

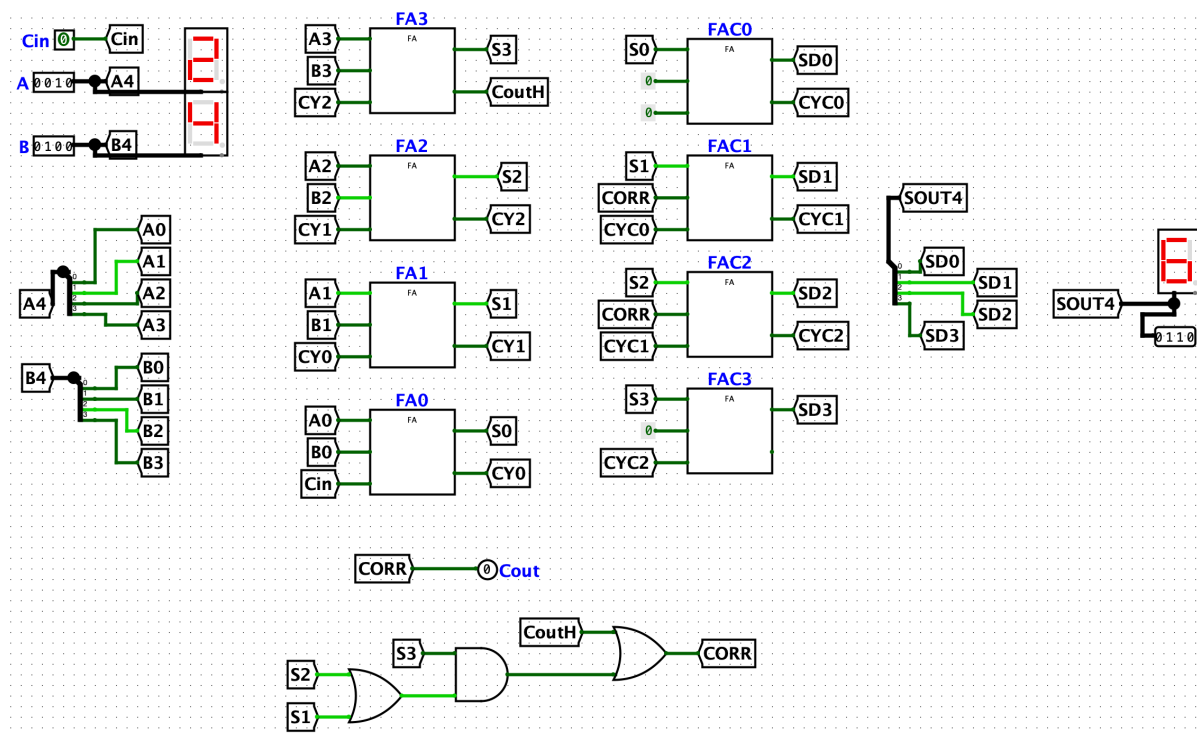
مرحله‌ی سوم: جمع تصحیحی $+6$. اگر $CORR = 1$ باشد، باید مقدار ۶ (باینری ۰۱۱۰) به عدد خام اضافه شود. چون بیت‌های ۰ و ۳ عدد ۶ صفر هستند، کافی است CORR فقط به بیت‌های ۱ و ۲ اضافه شود - بدون نیاز به جمع‌کننده یا گیت AND ۴ بیتی جداگانه. این کار با زنجیره‌ی دوم از ۴ تمام‌جمع‌کننده (FAC0 تا FAC3) انجام می‌شود:

نمونه	بیت	ورودی A	ورودی B	Cin	جمع نهایی	رقم نقلی
FAC0	۰	S_0	۰	۰	SD_0	CYC ₀
FAC1	۱	S_1	CORR	CYC ₀	SD_1	CYC ₁
FAC2	۲	S_2	CORR	CYC ₁	SD_2	CYC ₂
FAC3	۳	S_3	۰	CYC ₂	SD_3	(استفاده نمی‌شود)

بیت‌های $SD_0...SD_3$ (رقم نهایی و تصحیح‌شده) با یک تفکیک‌کننده ترکیبی به پایه‌ی خروجی S وصل می‌شوند و هم‌زمان برای نمایش بصری به یک نمایشگر هفت‌قطعه‌ای (Hex Digit Display) می‌روند.

چرا رقم نقلی FAC3 استفاده نمی‌شود؟

چون حداکثر مقدار ممکن در این جمع، $15 = 9 + 6$ است که همیشه در ۴ بیت جا می‌شود؛ بنابراین این جمع هرگز سرریز نمی‌کند.



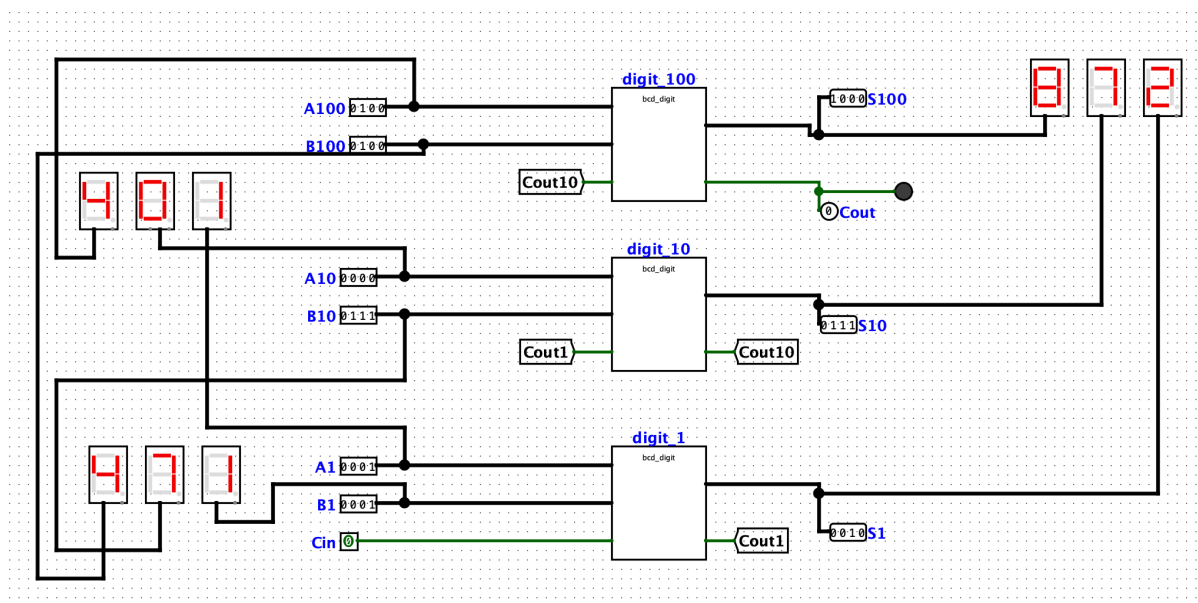
مدار اصلی

پایه	جهت	پهنا
A1, A10, A100	ورودی	۴ بیت هرکدام (رقم صدگان، دهگان، یکان عدد اول)
B1, B10, B100	ورودی	۴ بیت هرکدام (رقم صدگان، دهگان، یکان عدد دوم)
Cin	ورودی	۱ بیت
S1, S10, S100	خروجی	۴ بیت هرکدام
Cout	خروجی	۱ بیت

سه نمونه از جمع کننده ی یک رقمی دهدهی دقیقاً مانند جمع دستی چند رقمی روی کاغذ، به صورت زنجیره ای به هم وصل شده اند:

نمونه	رقم	A	B	Cin	Cout می رود به
digit_1	یکان	A1	B1	Cin (ورودی مدار)	تونل Cout1
digit_10	دهگان	A10	B10	تونل Cout1	تونل Cout10
digit_100	صدگان	A100	B100	تونل Cout10	پایه ی Cout مدار

هر رقم روی یک نمایشگر Hex Digit Display جداگانه نشان داده می شود و Cout نهایی به یک LED وصل است (در صورت سرریز از عدد ۹۹۹، روشن می شود).



نتایج تست

هر سه مدار با بردار تست و از طریق خط فرمان Logisim در حالت headless تست شدند:

```
CE103-CAL/Assignments/Lab01-BCD-Adder on / main [!+]  
● > ./run_tests.sh  
circ : bcd_adder.circ  
java : /opt/homebrew/opt/openjdk@21/bin/java  
jar : /Applications/Logisim-evolution.app/Contents/app/logisim-evolution-4.1.0-all.jar  
vecs : 3  
  
=== tests_bcd_digit.vec → circuit 'bcd_digit' ===  
Loading test vector "tests_bcd_digit.vec"...  
Running 200 vectors...  
200  
Passed: 200, Failed: 0  
PASS: tests_bcd_digit.vec  
  
=== tests_full_adder.vec → circuit 'full_adder' ===  
Loading test vector "tests_full_adder.vec"...  
Running 8 vectors...  
8  
Passed: 8, Failed: 0  
PASS: tests_full_adder.vec  
  
=== tests_main_smoke.vec → circuit 'main' ===  
Loading test vector "tests_main_smoke.vec"...  
Running 6 vectors...  
6  
Passed: 6, Failed: 0  
PASS: tests_main_smoke.vec  
  
summary: 3/3 passed, 0 failed  
(base)
```

بردارهای تست داخل فایل های *.vec قرار دارند.